

Your Thesis Title



Benjamin Williams

School of Computer Science

College of Science

Hogwarts School of Witchcraft and Wizardry

Submitted in partial satisfaction of the requirements for the
Degree of Doctor of Philosophy
in Computer Science

Supervisor Dr. Mantis Toboggan

July, 2025

A blank page

(***Todo:*** Note that you can use the `\todo` command to add notes like these.)

Here is an optional page environment you could use for things like:

- Your own custom preamble chapters (use `\chapterTitle` for titles!). If you want to add these to your table of contents, use the `\addChapterToTOC{text}` command and enable manual TOC entries in `preamble/thesis-details.tex` with `\enableManualTOCEntries`.
- “*This work is dedicated to...*”
- A copyright notice
- A declaration of authorship
- Additional notes
- Quotes
- List of publications
- An actual blank page

You can pass an optional parameter to this environment with the value `c` to centre this text vertically on the page.

Statement of Originality

The work presented in this thesis/dissertation is entirely from the studies of the individual student, except where otherwise stated. Where derivations are presented and the origin of the work is either wholly or in part from other sources, then full reference is given to the original author. This work has not been presented previously for any degree, nor is it at present under consideration by any other degree awarding body.

Statement of Availability

I hereby acknowledge the availability of any part of this thesis/dissertation for viewing, photocopying or incorporation into future studies, providing that full reference is given to the origins of any information contained herein. I further give permission for a copy of this work to be deposited to any repository authorised for use by Hogwarts School of Witchcraft and Wizardry. I acknowledge that Hogwarts School of Witchcraft and Wizardry may make the title and a summary of this thesis/dissertation freely available.

Signed: Benjamin Williams



Acknowledgements

Firstly, I want to thank somebody, and somebody else.¹ Here is another thing.

¹Here is a footnote

Abstract

Here is the abstract for this thesis.

List of Tables

3.1 Here is a table. The caption goes above like this. 7

List of Figures

3.1 A picture of a duck. 8

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Task	2
2	Fundamentals	5
2.1	Machine Learning	5
2.1.1	Face recognition tasks	5
	Verification	5
	Identification	5
2.1.2	Neural networks for vision	5
2.1.3	Embedding learning and similarity	5
2.2	Supervised learning	5
2.3	Performance metrics (verification)	5
2.4	Runtime metrics	5
2.5	Overview of models (architecture essentials)	5
2.6	Datasets and preprocessing	5
3	Methods	7
3.1	Requirements)	7
3.2	Model and Dataset Selection	7
3.3	Concept and design	7
3.3.1	GUI Application	7
3.3.2	Validation and Benchmarking Pipeline	7
3.3.3	Performance Evaluation (CPU vs GPU)	7
3.3.4	Custom Dataset Integration	7
4	Results and Discussions	9
4.1	Experimental Setup	9
4.2	Quantitative Results (Accuracy, AUC, EER, TAR@FAR, etc.)	9
4.3	ROC Analysis (Image vs Video)	9
4.4	Comparison of Models	9

4.5	Discussion and Observations	9
5	Conclusions	10
6	References	11

Chapter 1

Introduction

1.1 Motivation

Face recognition has become an integral component of many modern applications, ranging from smartphone authentication and social media photo tagging to access control and video surveillance. As these systems increasingly influence security-critical and privacy-sensitive domains, the reliability and robustness of the underlying face recognition models are of central importance. Small differences in model performance can have significant practical consequences, for example in terms of false acceptances, false rejections, or biased behaviour across different conditions.

In recent years, a large number of open-source face recognition models have been proposed, such as FaceNet, ArcFace, and AdaFace, many of which provide high accuracy on standard benchmarks. However, these models are often evaluated in isolation, using different experimental setups, datasets, and metrics. This makes it difficult for practitioners to understand the real trade-offs between them, especially when models must operate under realistic constraints such as limited computing resources or the need for real-time processing. A systematic and fair comparison of well-known models, implemented in a common framework such as PyTorch, is therefore needed to support informed model selection for practical applications.

Moreover, existing benchmark results are usually reported offline and do not provide an easy way to explore model behaviour interactively. Developers and researchers who want to test models on their own hardware or with their own data often need to

implement ad-hoc scripts and tools, which is time-consuming and error-prone. A user-friendly application that integrates state-of-the-art models, standard datasets, and real-time webcam input into a single interface would lower the barrier to entry and make it easier to study the relationship between accuracy, latency, and resource usage.

This thesis is motivated by the need for transparent, comparable, and practically relevant evaluation of face recognition models. By benchmarking several widely used PyTorch implementations on standard datasets and embedding them into an interactive runtime benchmarking application, the work aims to provide both a systematic empirical study and a tool that can support future research and real-world deployment.

1.2 Task

The central task of this thesis is to design and implement a framework for systematic benchmarking and real-time analysis of face recognition models. Building on existing PyTorch implementations of well-known open-source models, the work aims to provide both a thorough empirical comparison and a practical tool that can be used to explore their behaviour under realistic conditions.

Concretely, the thesis is structured around three main tasks:

- **Evaluation of face recognition models:** Implement a unified interface for several pretrained models (e.g. FaceNet, ArcFace, AdaFace) in PyTorch and evaluate their performance using standard verification benchmarks. This includes computing common metrics such as accuracy, precision, recall and ROC/AUC on datasets like LFW, CFP-FP and YTF.
- **Benchmarking of runtime performance:** Measure and compare the computational characteristics of the selected models, including latency per frame, achievable frame rate (FPS) and memory usage on the available hardware. The goal is to understand the trade-offs between recognition accuracy and real-time capability.

- **Development of a runtime benchmarking application:** Design and implement a PyQt-based desktop application that integrates the selected models and exposes their functionality through a user-friendly graphical interface. The application should allow users to choose a model, load images or enable a web-cam stream, and observe both recognition results and live performance metrics in real time.

By completing these tasks, the thesis will deliver (i) a consistent benchmarking setup for face recognition models, (ii) a comparative analysis of their accuracy and runtime properties, and (iii) a reusable application that can support further research and practical deployment decisions.

Command	Description	Example
<code>\thesisLogoPath</code>	Sets the path to the logo for the title.	<code>\thesisLogoPath{img/your-logo-here.jpg}</code>
<code>\thesisSubmissionDate</code>	Manually sets the submission date of the thesis, for the title page.	<code>\thesisSubmissionDate{July, 2019}</code>
<code>\thesisDegree</code>	Sets the name of the degree for this thesis.	<code>\thesisDegree{Doctor of Philosophy}</code>
<code>\thesisProgramme</code>	Sets the name of your degree programme/subject.	<code>\thesisProgramme{Biology}</code>
<code>\thesisSupervisor</code>	Sets the name of your supervisor (optional).	<code>\thesisSupervisor{Dr. Mantis Toboggan}</code>

<code>\thesisSecondSupervisor</code>	Sets the name of your second supervisor (optional).	<code>\thesisSecondSupervisor{Dr. Mantis Toboggan}</code>
<code>\thesisThirdSupervisor</code>	Sets the name of your third supervisor (optional).	<code>\thesisThirdSupervisor{Dr. Mantis Toboggan}</code>
<code>\thesisSchool</code>	Sets the name of your school.	<code>\thesisSchool{School of Chemistry}</code>
<code>\thesisCollege</code>	Sets the name of your college.	<code>\thesisCollege{College of Science}</code>
<code>\thesisUniversity</code>	Sets the name of your university.	<code>\thesisUniversity{University of Lincoln}</code>

In addition, `\thesisSubmissionText{text}` overrides the submission statement text, so you can type whatever you'd like onto the front page. This is optional, though. For example, if you wish to erase it entirely, you can use `\thesisSubmissionText{}`. Example usage can be found in `preamble/thesis-details.tex`.

Chapter 2

Fundamentals

2.1 Machine Learning

2.1.1 Face recognition tasks

Verification

Identification

2.1.2 Neural networks for vision

2.1.3 Embedding learning and similarity

2.2 Supervised learning

2.3 Performance metrics (verification)

2.4 Runtime metrics

2.5 Overview of models (architecture essentials)

2.6 Datasets and preprocessing

Machine Learning (ML) has advanced rapidly in recent years, driven by increasing computational power and the availability of large-scale datasets. Despite these de-

velopments, the performance and reliability of ML models depend heavily on the quality of the data on which they are trained and evaluated. High-quality datasets are essential because they capture the relevant characteristics of real-world phenomena, provide sufficient diversity and coverage, and minimize sources of bias or noise. When datasets are incomplete, inaccurate, or unrepresentative, model performance can degrade significantly, leading to reduced accuracy, poor generalization, and unintended biases during deployment. Understanding the dimensions of dataset quality and their influence on model behavior is therefore a crucial prerequisite for building effective and trustworthy ML systems [gong2023dataquality].

Chapter 3

Methods

3.1 Requirements)

3.2 Model and Dataset Selection

3.3 Concept and design

3.3.1 GUI Application

3.3.2 Validation and Benchmarking Pipeline

3.3.3 Performance Evaluation (CPU vs GPU)

3.3.4 Custom Dataset Integration

Here is a sentence, and you can see a nice picture in Figure 3.1. You can also use the `autoref` command to do this automatically, like this: Figure 3.1.

Also, a table can be found in Table 3.1. You should use a L^AT_EX table generator like <https://www.tablesgenerator.com/> if you want to make your life easier.

Table 3.1: Here is a table. The caption goes above like this.

First name	Last name	Age
Bob	Bobbington	24
Benth	Wavies	49
Joe	Bloggs	37
Billy	Bob	10



Figure 3.1: A picture of a duck.

Chapter 4

Results and Discussions

4.1 Experimental Setup

4.2 Quantitative Results (Accuracy, AUC, EER, TAR@FAR, etc.)

4.3 ROC Analysis (Image vs Video)

4.4 Comparison of Models

4.5 Discussion and Observations

Chapter 5

Conclusions

...

Chapter 6

References